Pontificia Universidad JAVERIANA Colombia

RAG conversacional y memoria

Semana 5 · Preguntas de seguimiento y memoria por conversación
Tópicos Avanzados en Analítica II – IA Generativa
Maestría en Analítica para la Inteligencia de Negocios

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso

Tópicos Avanzados en Analítica II

Estructura del curso

Fundamentos con LangChain
Semanas 1–3 ✓

RAG y evaluación
Semanas 4–6

Model Context Protocol
Semana 7

LangGraph y patrones agénticos
Semanas 8–11

LLMOps y despliegue
Semana 12

Semana 5 · RAG conversacional y memoria

- Por qué una pregunta de seguimiento falla
- Reformular con el historial antes de recuperar
- Multi-query para preguntas coloquiales
- Medir la recuperación: coseno, BM25 y RRF
- Un agente que escribe su propia búsqueda
- Memoria por `thread_id` con un checkpointer

Las ideas que traemos de la semana 4

Repaso

Antes de avanzar

1. La recuperación decide qué puede saber el modelo.
2. Cada dato de la respuesta se cita con [archivo, p. N].
3. «No está en los documentos» es una respuesta correcta.

Una conversación, no una pregunta

El problema de la semana

El retriever no lee el historial

«Tengo 45 días de mora en mi Crédito Libre Inversión. ¿Puedo reestructurarlo?»

«¿Y **cuántas veces** puedo hacerlo?»

La segunda pregunta solo tiene sentido con la primera.

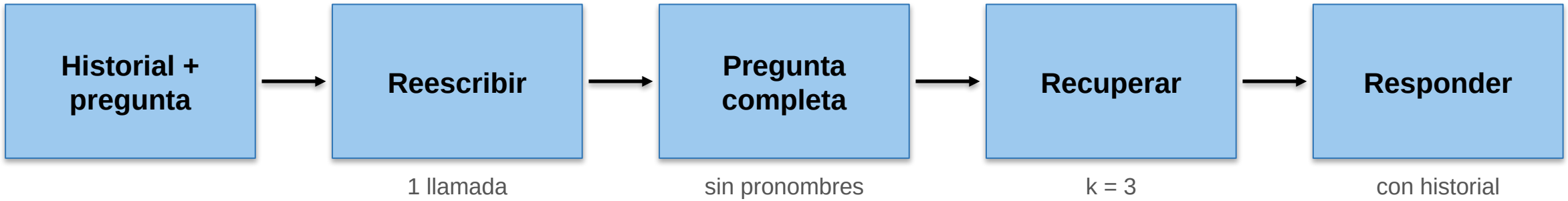
Pontificia Universidad JAVERIANA Colombia

Transformar la pregunta

Antes de recuperar

Reformular antes de recuperar

L09 · secciones 2 a 4



«¿Cuántas reestructuraciones puede solicitar un cliente de ACME con 45 días de mora…?»

La cadena conversacional

L09 · sección 4

MessagesPlaceholder

El historial va a la respuesta; la pregunta reescrita, al retriever.

```
qa_prompt = ChatPromptTemplate.from_messages([
    ("system", SYSTEM_PROMPT),
    MessagesPlaceholder("history"),
    ("human", "Fragmentos:\n\n{context}\n\nPregunta: {question}"),
])

@chain
def conversational_rag(inputs: dict):
    question, history = inputs["question"], inputs["history"]
    standalone_question = model.invoke(rewrite_prompt.format_messages(
        conversation=format_history(history), question=question)).text
    docs = retriever.invoke(standalone_question)
    answer = model.invoke(qa_prompt.format_messages(
        history=history, context=format_docs(docs), question=question))
    return {"standalone_question": standalone_question,
            "docs": docs, "answer": answer}
```

Varias consultas para una pregunta coloquial

L09 · secciones 5 y 6

Multi-query

1. «Me despidieron, ¿qué pasa con mis cuotas?» no usa las palabras de la política.
2. El modelo genera 3 versiones con salida estructurada: desempleo, pausar cuotas, mora.
3. Se recupera con cada una y se unen los fragmentos sin duplicados.
4. La cadena conversacional sigue diciendo «no está» cuando no está.

Lo que cuesta cada estrategia

L09 · sección 7

Estrategia	Llamadas por pregunta	Cuándo
Pregunta literal	1	La pregunta ya es completa
Reformular	2	Preguntas de seguimiento
Multi-query	2 + más recuperaciones	Preguntas coloquiales o ambiguas

Medir la recuperación

L09 · sección 8 · 19 preguntas con respuesta

Retriever	Página correcta en el top 3
Coseno (embeddings)	15 / 19
BM25 (palabras)	16 / 19
RRF (fusión de ambos)	17 / 19

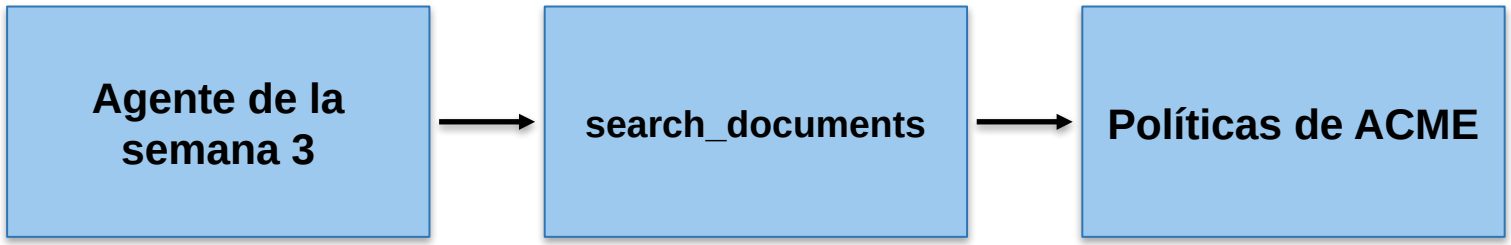
Pontificia Universidad JAVERIANA Colombia

Memoria

El agente recuerda la conversación

El retriever como herramienta

L10 · secciones 1 y 2



Ahora el agente decide cuándo buscar y escribe su propia consulta.

Memoria con un checkpointer

L10 · secciones 3 y 4

Un hilo por conversación

Sin checkpointer,
el agente pregunta
qué es «hacerlo».

```
from langgraph.checkpoint.memory import InMemorySaver

memory_agent = create_agent(
    model, tools=[search_documents],
    system_prompt=SYSTEM_PROMPT,
    checkpointer=InMemorySaver(),
)
config = {"configurable": {"thread_id": "c-1042-mora"}}
memory_agent.invoke({"messages": [HumanMessage(first)]}, config)
memory_agent.invoke({"messages": [HumanMessage(follow_up)]}, config)
# «Cada crédito admite una sola reestructuración por año
#   [acme_collections_policy.pdf, p. 2]»
```

Conversaciones aisladas

L10 · secciones 5 y 6

Hilo	Mensajes guardados	Qué responde
`c-1042-mora`	10	Conoce la mora y la reestructuración
`c-2210-nuevo`	2	Pregunta a qué se refiere «hacerlo»
Tokens de entrada	712 → 1302 → 1764 → 1883	El historial crece en cada turno

Pontificia Universidad JAVERIANA Colombia

Laboratorio

L09-ConversationalRAG · L10-Memory

¿Qué haremos?

Laboratorio

Nuestro recorrido

1. L09: ver fallar la pregunta de seguimiento y reformularla (8 llamadas).
2. L09: multi-query y la métrica de recuperación sin modelo.
3. L10: un agente sin memoria y con `InMemorySaver` (10 llamadas).
4. L10: leer el estado guardado y comparar dos hilos.

Las ideas que nos llevamos

Cierre

Criterios de construcción

1. El retriever busca con el texto exacto que recibe.
2. Reformular cuesta una llamada; recuperar mal cuesta la respuesta.
3. Medir la recuperación no requiere al modelo.
4. La memoria se separa por `thread_id` y crece con cada turno.

Para seguir leyendo

Lecturas de la semana

Fuente	Qué revisar
Learning LangChain, cap. 3 y 4	Transformar la consulta y memoria
Documentación de LangGraph	Persistence y checkpointers
Cormack et al. (2009)	Reciprocal Rank Fusion



Próxima semana

Evaluar el RAG de ACME con LangSmith y OpenEvals.
Notebook: `L11-RAGEvaluation.ipynb`

Sergio Alberto Mora Pardo - H1 2027
GitHub · LinkedIn · Repositorio del curso